LAB GUIDELINES

Lab 14 — High Availability: Clustering, Load Balancing, NIC Teaming

Maps to CompTIA Server+ objective **2.4** — Clustering (active-active, active-passive, failover, failback, heartbeat), Fault tolerance (server-level vs. component redundancy), Redundant server network infrastructure (load balancing — software/hardware, round-robin, MRU), NIC teaming and redundancy (failover, link aggregation).

Run all commands on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): `haproxy`, `keepalived`, `nginx`, `ifenslave`, `apache2-utils`

```
apt update && apt install -y haproxy keepalived nginx ifenslave apache2-utils
```

Step 1 — Two backend "servers" listening on different ports

```
mkdir -p /srv/back1 /srv/back2
echo "Backend 1" > /srv/back1/index.html
echo "Backend 2" > /srv/back2/index.html
nohup python3 -m http.server 8001 --directory /srv/back1 >/tmp/b1.log 2>&1 &
nohup python3 -m http.server 8002 --directory /srv/back2 >/tmp/b2.log 2>&1 &
sleep 1 && curl -s 127.0.0.1:8001 && curl -s 127.0.0.1:8002
```

Step 2 — HAProxy load balancer (round-robin)

```
cat > /etc/haproxy/haproxy.cfg <<'EOF'
global
    daemon
defaults
    mode http
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend in
    bind *:8080
    default_backend pool

backend pool
    balance roundrobin
    option httpchk GET /
    server b1 127.0.0.1:8001 check inter 2s fall 2 rise 2
    server b2 127.0.0.1:8002 check inter 2s fall 2 rise 2
EOF
systemctl restart haproxy
for i in 1 2 3 4; do curl -s 127.0.0.1:8080; done
```

You should see Backend 1 / Backend 2 alternating — round-robin distribution.

Step 3 — Algorithms: round-robin vs. least-conn vs. MRU

```
# Swap balance line, restart, re-test:
sed -i 's/balance roundrobin/balance leastconn/' /etc/haproxy/haproxy.cfg
systemctl restart haproxy
```

Server+ 2.4 lists **Round robin** and **Most recently used (MRU)** — MRU prefers the most recently used backend to maximise cache locality (the HAProxy equivalent is `source` or `last` hashing).

Step 4 — Failover test (active-passive at the backend level)

```
kill $(lsof -ti :8001) || pkill -f '8001'
for i in 1 2 3 4; do curl -s 127.0.0.1:8080; echo; done
```

```
# Only Backend 2 responds — HAProxy health-checks removed B1
curl -s 'http://127.0.0.1:8080/;csv' || systemctl status haproxy | head
```

This is **failover**. Restart the backend and watch **failback**:

```
nohup python3 -m http.server 8001 --directory /srv/back1 >/tmp/b1.log 2>&1 &
sleep 4
for i in 1 2 3 4; do curl -s 127.0.0.1:8080; echo; done
```

Step 5 — VRRP / heartbeat with keepalived (active-passive LB pair)

In production you run **two** HAProxy nodes and float a VIP between them. The config sketch:

```
cat > /etc/keepalived/keepalived.conf <<'EOF'
vrrp_instance VI_1 {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 150
    advert_int 1
    authentication { auth_type PASS; auth_pass labpass; }
    virtual_ipaddress { 10.99.99.99/24 }
}
EOF
echo "(keepalived needs two hosts to fully demo — config shown for reference)"
```

`advert_int 1` is the **heartbeat** every 1 s. If the master stops sending, the backup with the next-highest priority takes the VIP.

Step 6 — NIC teaming / bonding (link aggregation + failover)

```
modprobe bonding mode=active-backup miimon=100
ip link add bond0 type bond mode active-backup miimon 100
ip link set eth0 down
# ip link set eth0 master bond0    # would attach eth0 to bond0 on real hosts
ip -d link show bond0
```

Two modes that matter for the exam:

- **active-backup (mode 1)** = failover only, one NIC carries traffic at a time.
- **802.3ad LACP (mode 4)** = link aggregation, both NICs carry traffic, the switch must support LACP.

Step 7 — Patching cluster nodes correctly

Server+ 2.4 lists **proper patching procedures** under clustering. The pattern:

1. Drain the node from the load balancer (`server b1 ... disabled`).
2. Verify the remaining nodes can carry full load (capacity from Lab 12).
3. Patch, reboot, smoke-test.
4. Re-enable in the LB; watch health-check pass for one full interval.
5. Repeat for the next node.

Never patch all nodes in parallel — that defeats the cluster.

Step 8 — Server-level vs. component redundancy

Layer	Component redundancy	Server-level redundancy
Power	Dual PSU on one server	Two servers on separate UPS
Disk	RAID	Two servers with replication
Network	NIC teaming	Two servers behind LB + VIP
Compute	ECC RAM	N+1 cluster nodes

Component redundancy survives a *part* failure; server-level redundancy survives an *entire server* failure.

Free online tools

- **HAProxy docs** — https://www.haproxy.org/#docs
- **keepalived docs** — https://www.keepalived.org/manpage.html
- **Linux bonding HOWTO** — https://www.kernel.org/doc/Documentation/networking/bonding.txt

High availability with Kubernetes (bonus asset)

Kubernetes provides HA the same way HAProxy + keepalived do here, but at the orchestration layer: multiple replicas behind one load-balanced Service, with self-healing. Apply [`assets/k8s-ha-deployment.yaml`](assets/k8s-ha-deployment.yaml) on the **Killercoda Kubernetes** playground (https://killercoda.com/playgrounds/scenario/kubernetes):

```
kubectl apply -f k8s-ha-deployment.yaml
kubectl get pods -l app=ha-web -o wide
kubectl delete pod <one-pod-name>     # watch Kubernetes self-heal back to 3 replicas
kubectl get svc ha-web                 # the load-balanced virtual IP
```

What you learned

- Active-active vs. active-passive clustering shown with HAProxy + python backends.
- Round-robin / least-conn / MRU load-balancing algorithms.
- Failover + failback observed live.
- VRRP heartbeat with keepalived for an HA LB pair.
- NIC teaming: active-backup (failover) vs. 802.3ad (link aggregation).
- Cluster-aware patching procedure to avoid self-inflicted outages.